

# Tutorial Operativo Completo

Refactorización de Controllers, Jobs y Webhooks – Laravel

Incluye: tutorial paso a paso, PR real, checklist diaria y scaffolding automático.

## Introducción

Este documento es un tutorial práctico y operativo para refactorizar el repositorio Laravel, reduciendo el alto acoplamiento detectado en la Fase 0. Está diseñado para que cualquier desarrollador pueda seguirlo paso a paso sin tomar decisiones arquitectónicas adicionales.

# 1. Tutorial paso a paso – Controllers

Objetivo: convertir controllers grandes y acoplados en orquestadores delgados. Toda la lógica de negocio se mueve a Actions.

## Paso 1 – Crear FormRequest

app/Http/Requests/Memberships/CheckoutRequest.php

```
class CheckoutRequest extends FormRequest {
    public function rules(): array {
        return [
            'team_id' => 'required|exists:teams,id',
            'discount_code' => 'nullable|string',
        ];
    }
}
```

## Paso 2 – Crear DTO

app/Application/Memberships/DTO/CheckoutData.php

```
final class CheckoutData {
    public function __construct(
        public int $teamId,
        public ?string $discountCode
    ) {}

    public static function fromRequest(CheckoutRequest $r): self {
        return new self((int)$r->team_id, $r->discount_code);
    }
}
```

## Paso 3 – Crear Action

app/Application/Memberships/Actions/CheckoutAction.php

```
final class CheckoutAction {
    public function execute(User $user, CheckoutData $data) {
        $team = Team::findOrFail($data->teamId);
        if ($user->hasActiveMembershipInSeason($team->season_id)) {
            throw new DomainException();
        }
        // lógica real aquí
    }
}
```

## Paso 4 – Controller limpio

```
public function checkout(CheckoutRequest $request, CheckoutAction $action) {
    return $action->execute(
        auth()->user(),
        CheckoutData::fromRequest($request)
    );
}
```

## 2. Tutorial paso a paso – Jobs

Objetivo: convertir Jobs y Commands en wrappers sin lógica de negocio.

Antes:

```
class SyncStripeJob {  
    public function handle() {  
        if ($user->isPremium()) {  
            Stripe::charge(...);  
        }  
    }  
}
```

Después:

```
class SyncStripeJob {  
    public function handle(SyncStripeAction $action) {  
        $action->execute($this->userId);  
    }  
}
```

### 3. Checklist diaria para el equipo

- El Controller solo orquesta (sin negocio)
- Existe una Action clara y testeable
- No hay Stripe / OneSignal / Redis en Controller o Job
- Jobs no contienen reglas de negocio
- Se agregaron tests mínimos
- Cumple reglas arquitectónicas v1

## 4. Scaffolding automático (artisan)

Para acelerar el refactor, se recomienda crear comandos artisan que generen FormRequest, DTO y Action automáticamente.

```
php artisan make:action Memberships/CheckoutAction  
php artisan make:dto Memberships/CheckoutData  
php artisan make:request Memberships/CheckoutRequest
```

Estos comandos reducen errores humanos y aseguran consistencia en todo el repo.

## Cierre

Siguiendo este tutorial, el equipo puede refactorizar progresivamente el repositorio, reduciendo el acoplamiento sin reescribir el sistema. El patrón es repetible, seguro y escalable.